CIS 4020 · DATA STRUCTURES

Tables, Priority Queues, and Heaps: Overview and Objectives

The table ADT, the priority queue ADT, heaps, and heapsort

1. Overview

Most of the structures we have built so far were judged by how well they do everything: add, remove, search, and traverse. This unit looks at two ADTs that are defined by *what you are allowed to ask for*, and that choice decides which implementation is best.

A **table** (also called a dictionary or a map) is a value-oriented ADT. Every item has a **search key**, and the table is built to insert, delete, and retrieve items by that key quickly. Searching on anything other than the key means looking through the whole table. We compare the implementations you already know (unsorted and sorted arrays, unsorted and sorted linked lists, and binary search trees) and see that the best choice depends on which operations a program uses most. A binary search tree makes a natural table: each item in the tree is a row of the table.

A **priority queue** gives every item a priority value, and the only item you may remove is the one with the **highest priority**. Emergency room triage, a printer queue, and an operating system choosing which program runs next all work this way. A sorted array, a sorted linked list, and a binary search tree can each hold a priority queue, but each one makes either insertion or deletion slow, or depends on the shape of the tree.

Most of the unit is about the structure that fixes this: the **heap**. A heap is a complete binary tree in which every node has a priority at least as high as its children. In a **maxheap** the largest key is at the root, and in a **minheap** the smallest key is. Because a heap is always complete, it is always as short as a binary tree can be, so inserting and removing both take $O(\log n)$ time, no matter what order the items arrive in. Being complete also means a heap fits neatly in an array with no gaps, and simple arithmetic finds a node's parent and children.

You will learn to insert into a heap (place the item as the last node and trickle it up), to remove from a heap (remove the root, move the last node to the root, and trickle it down), and to change the key of an item already in the heap. We build the heap two ways, **array-based** and **reference-based**, and as a maxheap and a minheap, and then wrap it in a priority queue class. The unit ends with **heapsort**, which uses the same trickle down step to sort an array in place in $O(n \log n)$ time.

Several earlier units come back here. **Trees** supply the vocabulary (complete binary tree, parent, child, leaf, height), and the binary search tree becomes one of our table implementations. **Algorithm analysis** lets us compare the implementations fairly, and the **sorting** unit gives us mergesort and quicksort to measure heapsort against. The interface and wall idea from **stacks and queues** returns when the priority queue hides the heap underneath it.

Heaps are drawn by hand: we build a heap one insertion at a time, take it apart one deletion at a time, and look at the code that does the same work. Any code is shown in both Java (CIS) and C/C++ (CS).

2. Learning objectives

By the end of this unit you should be able to:

Remember and understand

- Define the table ADT and its search key, list its operations, and explain why a search on the key is faster than a search on any other field.
- Describe how a table can be stored in an unsorted array, a sorted array, an unsorted linked list, a sorted linked list, and a binary search tree.
- Define the priority queue ADT, list its operations, and give examples of programs that use one.
- Explain where the highest priority item sits when a priority queue is stored in a sorted array, a sorted linked list, and a binary search tree.
- Define a heap as a complete binary tree, and state the maxheap and minheap properties recursively.
- Explain how a heap differs from a binary search tree, including why the two children of a node may appear in either order.
- Identify the root, the last node, and the next open position in a heap.
- State the array index formulas for the parent, left child, and right child of a node, and explain why a complete tree leaves no gaps in the array.
- Describe the steps of heap insertion (place as the last node, trickle up) and heap deletion (remove the root, place the last node as the root, trickle down).
- Explain how a reference-based heap finds its last node and the position for a new node.
- Describe the two phases of heapsort: building a heap, then repeatedly moving the root to the end of the array.

Apply

- Draw the maxheap or minheap that results from inserting a sequence of keys into an empty heap, showing each trickle up.
- Draw the heap after each of a sequence of deletions, showing each trickle down.
- Change the key of an item in a heap and restore the heap, choosing correctly between trickling up and trickling down.
- Convert a heap between its tree drawing and its array form, in both directions.
- Trace heapsort on a small array, showing the array after the heap is built and after each step of the second phase.
- Trace the Java or C++ heap code by hand for a given sequence of operations, recording the array (or the tree) after each call.
- Write or complete a heap method, such as insert, remove, or change, in Java or C++, for an array-based or a reference-based heap.

Analyze

- Determine the time complexity of each table operation for each table implementation, and of insertion and deletion for each priority queue implementation.
- Explain why a heap with n items has height $\lceil \log_2(n+1) \rceil$, and use that to show that heap insertion and deletion take $O(\log n)$ time.
- Compare the array-based and reference-based heaps in memory use, size limits, and the work needed to find the last node.
- Compare heapsort with mergesort and quicksort in running time and extra memory.
- Decide whether a given tree or array is a valid maxheap or minheap, and point to each place where it fails.

Evaluate

- Choose a table implementation for a described program, based on how often it inserts, deletes, retrieves, and traverses, and justify the choice.
- Choose between a sorted list, a binary search tree, and a heap for a described priority queue, and justify the choice.
- Judge whether a given heap implementation is correct. Find errors such as trickling toward the wrong child, an off-by-one index or bounds check, a comparison that builds a minheap when a maxheap was intended, or a last node that is not removed from the tree.